HAI913I · Évolution et restructuration des logiciels

TP 1 — Analyse statique

Année 2026–2027 Enseignant Bachar Rima

OBJECTIF

Construire un **analyseur statique de programmes Java** : un outil en ligne de commande qui lit le code source d'un projet, en extrait la structure et les appels de méthodes, calcule des **métriques** et construit le **graphe d'appel**. Le CM 02 a présenté la chaîne *extracteur de modèle* → *modèle* → *extracteurs de propriétés* → *propriétés* → *traitements* ; ce TP la fait écrire, avec Eclipse JDT.
Chaque étape se termine par un **point de contrôle** sur un projet de validation fourni : vous savez à tout moment si votre outil compte juste.

PRÉREQUIS

- **CM 02** — AST, patron Visitor, *bindings*, type statique du receveur, graphe d'appel et ses limites (réflexion, injection de dépendances, appels virtuels).
- **CM 01** — pourquoi l'évolution d'un logiciel commence par sa compréhension, et ce qu'une métrique permet (ou non) d'affirmer.
- **Java et POO** — héritage, interfaces, surcharge, redéfinition, classes imbriquées, lambdas ; bases de Maven.

1. Ressources fournies

Toutes les ressources sont sur Moodle.

PROJETS À ANALYSER

- `resources/validation` — le **projet de validation** : 14 fichiers, 4 paquetages, une petite médiathèque (héritage, interfaces, énumération, classe imbriquée, surcharges, appels statiques, virtuels, récursifs, lambda). Il compile et s'exécute seul (`mvn -q compile exec:java`). **Tous les points de contrôle portent sur lui.**
- `resources/robustness` — trois échantillons volontairement imparfaits : dépendance manquante (`missing-dep`), erreur de syntaxe (`syntax-error`), dossier sans fichier Java (`no-java-files`).

POINT DE DÉPART ET ANNEXES

- `starter` (facultatif) — un squelette Maven qui compile, configure `ASTParser` pour résoudre les *bindings* et vérifie l'environnement (étape A0).
- **Annexe 1 « Du code source au graphe d'appel »** — l'architecture conseillée, composant par composant, dans le vocabulaire du CM 02.
- **Annexe 2 « Eclipse JDT et Spoon »** — configuration d'`ASTParser`, nœuds utiles, *bindings*, positions, pannes fréquentes et leur diagnostic ; Spoon en seconde partie.
- **Annexe 3 « Le patron Visitor, ASTVisitor et Spoon »** et `resources/visitor-examples` — du patron du GoF aux visiteurs de JDT et de Spoon, avec des exemples exécutables.
- `resources/spoon-tostring` (facultatif, partie C) — un processeur Spoon qui génère des méthodes `toString()`.

DOCUMENTATION DE RÉFÉRENCE

- Javadoc officielle de JDT Core, en particulier `ASTParser`, `ASTVisitor` et `CompilationUnit` ; le guide *Manipulating Java code*.
- Documentation de Spoon : spoon.gforge.inria.fr.

La liste commentée des ressources en ligne (obligatoires, utiles, facultatives) est à la fin de l'annexe JDT/Spoon.

BESOIN D'UN RAPPEL JAVA/POO ?

Si les bases du langage ou de la conception objet ne sont pas encore consolidées, deux listes de lecture sont à votre disposition : **Java/POO — partie 1** et **partie 2**.

2. Environnement technique

ÉLÉMENT	EXIGENCE POUR CE TP
JDK	**Java 17 au minimum**. Toutes les ressources sont validées sur **17.0.10** et **21.0.5**. JDT Core 3.46.0 exige lui-même Java 17.
Construction	**Maven** recommandé (validé avec **3.9.6**). Gradle est accepté si le README donne des commandes équivalentes et testées.
Analyse (requis)	**Eclipse JDT Core** `org.eclipse.jdt:org.eclipse.jdt.core:3.46.0`, depuis Maven Central. Ses dépendances Eclipse sont téléchargées automatiquement : **aucun JAR à installer à la main**.
Analyse (bonus)	**Spoon** `fr.inria.gforge.spoon:spoon-core:11.5.0`, qui s'appuie sur la même version de JDT Core.
Projet	Java 17, **sur le *classpath***; pas de `module-info.java`.
Interface	une **ligne de commande**. Aucune interface graphique n'est requise.
Visualisation	facultative (par exemple Graphviz pour convertir un fichier DOT en image) ; indiquez alors la version utilisée dans le README.

Environnement de développement. Il est **fortement recommandé** d'utiliser un environnement de développement intégré (IDE), notamment **Eclipse IDE** ou **IntelliJ IDEA** : navigation dans un code que l'on ne connaît pas, débogueur, import de projets Maven. Ce sont des compétences directement utiles à la compréhension de programmes, sujet de cette UE.

Il reste tout à fait possible de réaliser le TP avec un **éditeur de texte/code et les outils en ligne de commande** : Visual Studio Code, Sublime Text, Vim, Emacs, Xcode ou tout environnement équivalent, combinés à `mvn`, `javac` et `java`. Aucun élément du sujet ne dépend d'un IDE, et les deux approches sont évaluées de la même façon.

NOTE

Quel que soit votre choix, vous devez pouvoir expliquer *ce que votre outil fait à votre place* : qui résout les dépendances, où sont compilées les classes, ce que contient le *classpath*, quelle commande est lancée quand vous cliquez sur « Run », et depuis quel **répertoire de travail**.

3. Modalités de travail

Travail en binôme. Le TP 1 se réalise et se rend **en binôme**. Le travail en monôme reste possible **à titre exceptionnel**, pour qui n'a pas pu intégrer un binôme ; prévenez alors l'enseignant. Dans un binôme, **chacun des deux membres** doit pouvoir expliquer l'ensemble du rendu, et pas seulement « sa » partie. Discuter d'un problème avec d'autres binômes est normal ; rendre du code que l'on ne saurait pas expliquer ne l'est pas.

3.1. Usage des outils d'intelligence artificielle générative

L'usage d'outils d'IA générative est **autorisé**, à condition qu'il reste un *appui* et qu'il soit *déclaré*.

USAGES LÉGITIMES	CE QUI N'EST PAS ACCEPTABLE
Faire expliciter une notion du cours (AST, *binding*, type statique).	Faire produire tout ou partie de l'analyseur sans le comprendre.
Demander des exemples supplémentaires.	Rendre du code que vous ne sauriez pas justifier.
Comprendre un message d'erreur, chercher pourquoi un *binding* est nul.	Présenter comme vôtres des conventions ou une conception que vous n'avez pas examinées.
Discuter des alternatives de conception, organiser des idées, améliorer la rédaction d'une explication, localiser une documentation.	Produire un compte-rendu dont le contenu ne correspond pas à votre code ou à votre vidéo.

Vous restez **entièrement responsables** de l'architecture, de l'implémentation, de la correction des résultats, de leur vérification et des explications que vous rendez. Vous devez pouvoir **expliquer et défendre** chaque élément significatif de votre travail.

> CE SUR QUOI PORTE RÉELLEMENT L'ÉVALUATION
> Il n'existe aucun moyen fiable de détecter automatiquement du texte ou du code engendré par une IA, et ce TP ne cherche pas à le faire. L'évaluation porte sur la **transparence**, la **compréhension démontrée**, la **cohérence entre le compte-rendu, le code et la vidéo**, votre capacité à **justifier vos choix** et la **reproductibilité** de votre travail. Un dossier cohérent et assumé, avec une déclaration honnête, n'est jamais pénalisé par le fait d'avoir utilisé un assistant.

Le compte-rendu comporte une section dédiée : voir § 5, point **A.7**.

4. Conventions communes

Une métrique n'a de sens qu'avec sa définition. Les conventions ci-dessous sont **imposées** : ce sont elles qui rendent les points de contrôle vérifiables et les résultats comparables d'un binôme à l'autre. Vous pouvez calculer d'autres variantes en plus, à condition de les nommer et de les définir.

4.1. Ce que l'on compte

NOTION	DÉFINITION POUR LE TP
Paquetage	chaque nom de paquetage déclaré distinct ; les fichiers sans déclaration `package` forment un paquetage « (défaut) ».
Classe (type)	toute déclaration **nommée** de classe, interface, énumération ou *record*, de premier niveau ou **membre** (imbriquée). Ne comptent pas : les classes anonymes, les classes locales (déclarées dans un corps de méthode), les types d'annotation. Nom qualifié avec des points : `library.service.LoanService.Loan`.
Attribut	chaque variable déclarée dans une déclaration de champ (`int a, b;` compte pour deux), statique ou non. Ne comptent pas : les constantes d'énumération, les composants d'un *record*.

NOTION (SUITE)	DÉFINITION POUR LE TP
Méthode	chaque méthode ou **constructeur écrit dans le code** d'un type compté, y compris abstrait ou d'interface. Ne comptent pas : les blocs d'initialisation, les membres implicites (constructeur par défaut, accesseurs d'un *record*, `values()`), les méthodes des classes anonymes.
Paramètres	nombre de paramètres écrits dans la déclaration (`String...` compte pour un).
Visibilité	`public`, `protected`, `private`, ou `package` en l'absence de modificateur (un membre d'interface peut être indiqué « public implicite »).
Superclasses	la chaîne complète, de la superclasse directe à la plus lointaine, **sans** `java.lang.Object`, y compris les classes de bibliothèque (`java.lang.Exception`…). Les interfaces se listent à part.

4.2. Lignes de code

- **Lignes de l'application** (question 2) : nombre total de lignes physiques des fichiers `.java` analysés, lignes vides et commentaires compris — ce que compte `Files.readAllLines(chemin).size()`.
- **Lignes d'une méthode** (questions 6 et 12) : nombre de lignes physiques couvertes par son **corps**, de la ligne de l'accolade ouvrante à celle de l'accolade fermante incluses ; **0** pour une méthode sans corps. La signature, la Javadoc et les annotations ne comptent pas.

4.3. « Les 10 % » et le seuil X

- Parmi n candidats, on retient les $k = \lceil n/10 \rceil$ premiers dans l'ordre **décroissant** de la valeur mesurée (au moins un dès que $n \geq 1$; aucun si $n = 0$).
- Tout candidat dont la valeur égale celle du k-ième est retenu aussi. Le résultat peut donc dépasser k éléments.
- Un candidat dont la valeur vaut **0** n'est jamais retenu.
- Questions 8 et 9 : les candidats sont **toutes les classes** du projet. Question 12 : le calcul est fait **classe par classe**, les candidats étant les méthodes de la classe.
- Question 11 : classes ayant **strictement plus** de X méthodes ; X est un entier positif ou nul fourni sur la ligne de commande. Une valeur mal formée est refusée avec un message clair.
- Les moyennes s'affichent avec deux décimales.

4.4. Appels, receveur et graphe d'appel

APPELS PRIS EN COMPTE

Un **appel** est un nœud `MethodInvocation` (`a.m()`, `m()`, `Type.m()`) ou `SuperMethodInvocation` (`super.m()`) situé dans le corps d'une méthode comptée. Les appels écrits dans une lambda ou dans une classe anonyme sont attribués à la méthode comptée qui les contient. Les références de méthode (`Type::m`) ne sont pas des appels. Les appels situés hors de toute méthode (initialiseur de champ) sont ignorés.

Type statique du receveur. Receveur explicite : type statique de l'expression (`loanable.checkOut()` → le type *déclaré* de `loanable`). Sans receveur : type qui déclare la méthode si elle est statique, sinon type englobant (`this` implicite). `super.m()` : superclasse directe du type englobant. Un type générique est désigné par son effacement (`java.util.List`).

SÉMANTIQUE DU GRAPHE D'APPEL

- **Nœuds** : les méthodes comptées du projet, identifiées par un nom stable et unique, par exemple `library.service.Catalog#add(library.model.Item)` ; les surcharges sont donc des nœuds distincts.
- **Arc** $f \to g$: le corps de f contient au moins un appel dont la **cible résolue à la compilation** (le *binding* de méthode : la déclaration choisie d'après le type statique du receveur) est g. Un appel à une méthode héritée vise la classe qui la déclare ; un appel via une interface vise la méthode de l'interface.
- **Pas d'expansion des appels virtuels** : aucune analyse de hiérarchie (CHA, RTA…) n'est demandée ; c'est une piste de bonus.
- Plusieurs sites d'appel de f vers g : **un seul arc**, dont le **poids** est le nombre de sites. Récursion : arc $f \to f$.
- Appels vers une méthode **extérieure** au projet (JDK, bibliothèques) et appels **non résolus** : pas d'arc, mais **comptés** et signalés.
- Constructeurs : nœuds du graphe (ce sont des méthodes comptées). Les arcs vers les constructeurs (`new`, `this(. . .)`, `super(. . .)`) sont facultatifs ; si vous les ajoutez, faites-en une option et indiquez-le.

CE QUE CE GRAPHE NE CONTIENT PAS

Ce graphe est une **approximation statique** (CM 02) : il ignore les appels par réflexion, les proxys, l'injection de dépendances, et ne dit pas quelle redéfinition sera exécutée. Il répond à la question « f nomme-t-il g dans son code ? ». Vous discuterez ces limites dans le compte-rendu.

PARTIE A Construire l'extracteur de faits

OBLIGATOIRE · COMPTE POUR LA NOTE SUR 20

Cette partie produit, à partir du code source, les **faits** dont toute la suite a besoin : la structure du projet et ses appels. Elle se construit étape par étape ; **n'attaquez pas une étape tant que le point de contrôle de la précédente n'est pas franchi**. Une métrique fausse vient presque toujours d'un fait mal extrait.

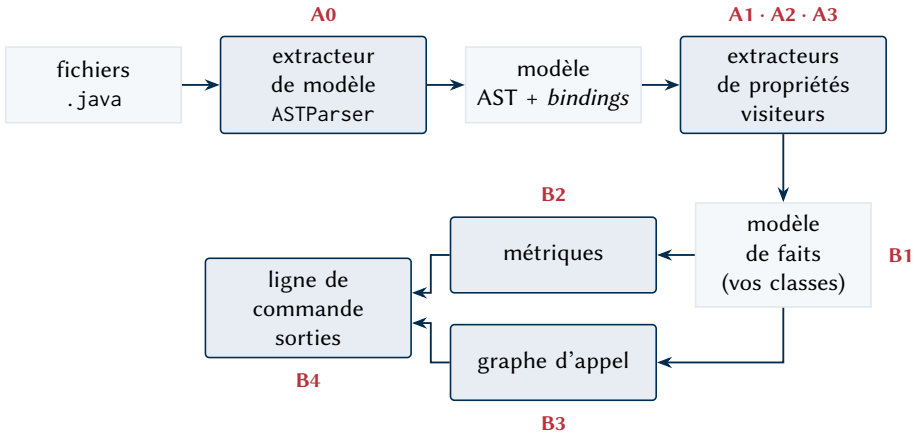


Figure 1 – La chaîne d'analyse du TP, dans le vocabulaire du CM 02. Les pastilles renvoient aux étapes.

A0 Environnement et projet de validation

TRAVAIL DEMANDÉ

1. Vérifiez `java -version` (17 ou plus) et `mvn -v`.
2. Compilez et exécutez le projet de validation : dans `resources/validation`, `mvn -q compile exec:java`. Lisez son code : vous allez l'analyser pendant tout le TP.
3. Créez votre projet d'analyseur : partez du `starter`, ou d'un projet Maven vide dépendant de JDT Core 3.46.0.
4. Faites analyser le projet de validation par `ASTParser` et affichez le nombre d'unités de compilation et d'erreurs de compilation.

```
$ cd starter
$ mvn clean package
$ java -jar target/hai913i-tp1-analyzer.jar ../resources/validation
Racine des sources     : ...\resources\validation\src
Unites de compilation  : 14
Erreurs de compilation : 0
```

INDICE

Les *bindings* ne se résolvent que si le parseur connaît la **racine des sources** et le JDK : c'est le rôle de `setEnvironment`. Sans cet appel, `createAST` échoue avec `IllegalStateException: Missing system library`. Sous Windows, entourez de guillemets les chemins contenant des espaces. Tout est détaillé dans l'annexe JDT/Spoon, section « Configurer `ASTParser` ».

POINT DE CONTRÔLE A0

Votre programme annonce **14 unités de compilation** et **0 erreur** sur le projet de validation. Si vous utilisez le `starter`, `mvn clean test` passe : son test vérifie aussi qu'un appel d'un fichier vers une méthode déclarée dans un autre fichier est bien résolu.

A1 Parcourir un AST

TRAVAIL DEMANDÉ

1. Écrivez un visiteur (sous-classe d'`ASTVisitor`) qui affiche l'arbre d'un fichier : le type de chaque nœud, indenté selon sa profondeur.
2. Sur `library/model/Dvd.java`, repérez les nœuds qui représentent la classe, ses attributs, ses méthodes et un appel. Faites de même sur `Category.java`.
3. Expliquez dans le compte-rendu le rôle de la valeur renvoyée par `visit(...)`, et ce que change `return false`.

INDICE

`preVisit2(ASTNode)` et `postVisit(ASTNode)` sont appelées pour *tous* les nœuds : pratiques pour suivre la profondeur. Sous Eclipse, la vue AST View montre le même arbre de façon interactive. L'annexe Visitor explique le lien entre le patron du GoF et `ASTVisitor`.

POINT DE CONTRÔLE A1

Pour `Dvd.java`, l'arbre montre un `TypeDeclaration`, un `FieldDeclaration` et trois `MethodDeclaration` (dont le constructeur, qui contient un `SuperConstructorInvocation`). Pour `Category.java`, la racine du type est un

`EnumDeclaration` : **`visit(TypeDeclaration)` ne le voit pas**. Retenez-le pour l'étape suivante.

A2 Extraire la structure du projet

TRAVAIL DEMANDÉ

Pour **chaque classe** du projet (au sens de § 4.1), extrayez :

1. son nom qualifié, son genre (classe, interface, énumération, *record*) et son paquetage ;
2. **toutes** ses superclasses et ses interfaces directes ;
3. ses attributs : nom, type déclaré, visibilité ;
4. ses méthodes : nom, nombre de paramètres, constructeur ou non.

INDICE

Lisez les membres d'un type dans `bodyDeclarations()` plutôt que par une visite récursive : sinon les méthodes de `LoanService.Loan` seront attribuées à `LoanService`. La chaîne des superclasses n'est pas écrite dans le fichier analysé : elle s'obtient par les *bindings* (`ITypeBinding.getSuperclass()`).

POINT DE CONTRÔLE A2

- **15 classes** : 12 classes, 2 interfaces, 1 énumération ; **59 méthodes**, dont 9 constructeurs ; **21 attributs**.
- `LoanService` : 6 méthodes et 5 attributs ; sa classe membre `LoanService.Loan` : 4 méthodes et 3 attributs.
- Superclasses de `library.service.LoanException` : `java.lang.Exception`, `java.lang.Throwable`.
- Attributs de `library.model.Item` : `MAX_LOAN_DAYS` (public), `id` et `title` (protected), `borrower` (private), `category` (package).

A3 Extraire les appels et le type statique du receveur

TRAVAIL DEMANDÉ

Pour chaque méthode de chaque classe, listez les **méthodes appelées** (§ 4.4) avec, pour chacune : son nom, le **type statique du receveur**, la méthode cible résolue (ou la mention « non résolue ») et la ligne de l'appel.

INDICE

`MethodInvocation.resolveMethodBinding()` donne la cible ; `getMethodDeclaration()` remonte de `List<Item>.add(Item)` à la déclaration générique. Le type du receveur est celui de l'expression : `getExpression().resolveTypeBinding()`. Un *binding* nul n'est pas une fatalité : l'annexe JDT/Spoon donne la liste des causes, mesurées sur ce projet.

POINT DE CONTRÔLE A3

- **94 appels** (93 `MethodInvocation` et 1 `SuperMethodInvocation`) : 46 vers des méthodes du projet, 48 vers l'extérieur, **0 non résolu**.
- `LoanService.borrow`, ligne 36, `loanable.checkOut(member)` : receveur `library.model.Loanable` — **pas** `Item`, même si l'objet réel en est un.
- `LibraryApp.main`, ligne 32, `zadig.getTitle()` : receveur `library.model.Book`, cible `library.model.Item#getTitle()` (méthode héritée).
- `Book.describe`, ligne 32, `super.describe()` : receveur `library.model.Item`. `Catalog.available`, ligne 37, dans une lambda, `item.isAvailable()` : receveur `library.model.Item`.
- Sur `resources/robustness/missing-dep` : l'appel à `StringUtils.capitalize` est signalé non résolu, l'appel à `formatter.decorate` est résolu, et l'analyse va jusqu'au bout.

PARTIE B Exploiter les faits : métriques et graphe d'appel

OBLIGATOIRE · COMPTE POUR LA NOTE SUR 20

La partie B ne relit plus le code source : elle travaille sur les faits extraits en partie A.

B1 Un modèle de faits indépendant de JDT

TRAVAIL DEMANDÉ

1. Représentez les faits par **vos propres classes** (types, attributs, méthodes, appels…), construites en une fois à partir de **toutes** les unités de compilation.
2. Les étapes B2 et B3 ne manipulent que ce modèle : aucune classe de `org.eclipse.jdt` n'y apparaît.
3. Justifiez vos choix de représentation dans le compte-rendu : identifiants des méthodes, traitement des surcharges, des classes imbriquées et des appels non résolus.

INDICE

Une analyse lancée deux fois sur le même projet doit produire exactement la même sortie : triez vos collections (ordre des fichiers, des types, des méthodes). L'annexe « du code source au graphe d'appel » propose une organisation en paquetages.

POINT DE CONTRÔLE B1

Les chiffres de A2 et A3 sont retrouvés **à partir du modèle**, sans réanalyser le code ; deux exécutions successives donnent une sortie identique. Les appels d'une classe vers une classe d'un autre fichier (par exemple de `LoanService` vers `Catalog`) sont résolus.

B2 Métriques

TRAVAIL DEMANDÉ

Calculez, avec les conventions de § 4 :

Q.	MÉTRIQUE
1	nombre de classes
2	nombre de lignes de code de l'application
3	nombre total de méthodes
4	nombre total de paquetages
5	nombre moyen de méthodes par classe
6	nombre moyen de lignes de code par méthode (méthodes ayant un corps)
7	nombre moyen d'attributs par classe
8	les 10 % de classes qui possèdent le plus de méthodes
9	les 10 % de classes qui possèdent le plus d'attributs
10	les classes qui font partie à la fois des deux catégories précédentes
11	les classes qui possèdent plus de X méthodes (X donné sur la ligne de commande)
12	pour chaque classe, les 10 % de ses méthodes qui possèdent le plus de lignes de code
13	le nombre maximal de paramètres parmi toutes les méthodes, et les méthodes concernées

INDICE

$\lceil n/10 \rceil$ en entiers : `(n + 9) / 10`. Un transtypage `(int) (n * 0.1)` arrondit vers le bas et donne 1 au lieu de 2 pour $n = 15$. Pour les lignes d'un corps : `unit.getLineNumber(body.getStartPosition())` et la position de son dernier caractère.

POINT DE CONTRÔLE B2 — PROJET DE VALIDATION

Q.	VALEUR ATTENDUE
1–4	15 classes · 493 lignes · 59 méthodes · 4 paquetages
5–7	3,93 méthodes par classe · 4,64 lignes par méthode (246 lignes pour 53 méthodes ayant un corps) · 1,40 attribut par classe
8	`Item` (10), puis `Member` et `LoanService` (6) : 3 classes pour $k = 2$
9, 10	`Item` et `LoanService` (5 attributs chacune) ; intersection : les mêmes
11	X = 5 : `Item`, `Member`, `LoanService` · X = 4 : 6 classes
12	`Item` : `checkOut` (7) · `LoanService` : `borrow` (14) · `Member` : ses 6 méthodes (3 lignes chacune) · `Loanable` : aucune (toutes à 0)
13	5 paramètres : `Statistics#report`

B3 Graphe d'appel

TRAVAIL DEMANDÉ

1. Construisez le graphe d'appel du projet à partir du modèle de faits, avec la sémantique de § 4.4.
2. Produisez une **sortie exploitable** : liste des arcs avec leur poids, et nombres de nœuds, d'arcs, d'appels externes et non résolus. Le format est libre (texte, CSV, JSON, DOT, GraphML…) : justifiez-le.
3. Permettez d'interroger le graphe dans les deux sens pour une méthode donnée : ses **appelées** et ses **appelantes**. Le graphe inversé est celui de l'analyse d'impact (CM 02).

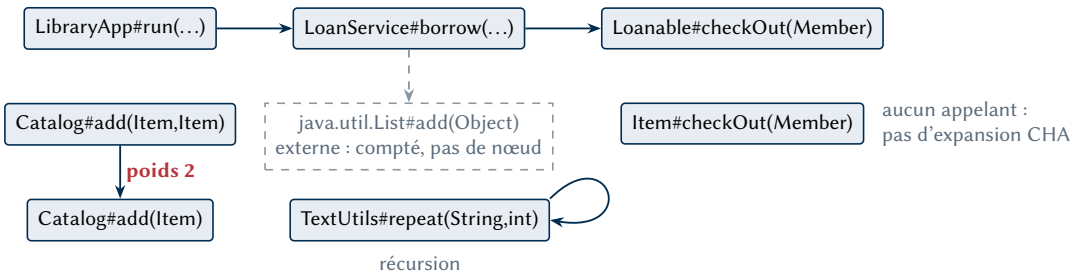


Figure 2 – Extrait du graphe d'appel du projet de validation.

POINT DE CONTRÔLE B3

- **59 nœuds**, **44 arcs**, **46 sites d'appel** internes, 48 appels externes, 0 appel non résolu.
- `Catalog#add(Item,Item)` → `Catalog#add(Item)` de poids 2 ; `TextUtils#repeat` s'appelle lui-même.
- Appelantes de `Loanable#checkOut(Member)` : `LoanService#borrow` seulement ; `Item#checkOut(Member)` n'a **aucune** appelante.
- Si vous ajoutez les arcs vers les constructeurs : 52 arcs et 56 sites.

B4 Un outil utilisable et reproductible

TRAVAIL DEMANDÉ

1. Offrez une **ligne de commande** qui prend au minimum : le chemin du projet à analyser, ce qu'il faut produire (structure, appels, métriques, graphe…) et le seuil X. La syntaxe est libre ; elle est documentée par le README et par un message d'usage.
2. Traitez les cas d'erreur : dossier inexistant, dossier sans fichier Java, X absent ou mal formé, fichiers qui ne compilent pas, *bindings* non résolus. Aucune pile d'exception brute : un message explicite et un code de sortie non nul lorsque l'analyse est impossible.
3. Vérifiez que votre projet se construit depuis un **état propre** (`mvn clean package` sur une copie fraîche, sans IDE) et s'exécute sur un chemin contenant des espaces.

INDICE

Un `main` de mille lignes n'est pas une architecture : la ligne de commande lit les arguments et délègue à des composants séparés (analyse, faits, métriques, graphe, affichage). Il faut pouvoir expliquer et tester chaque partie séparément. Le temps d'exécution n'est pas évalué : quelques secondes suffisent pour le projet de validation.

POINT DE CONTRÔLE B4

- `robustness/no-java-files` et un chemin inexistant : message clair, code de sortie non nul.
- `robustness/syntax-error` : les erreurs de `Broken.java` sont signalées, `Helper.java` est analysé, l'outil ne s'arrête pas.
- X = `abc` : refus explicite.
- Sur une copie fraîche de votre dépôt, les commandes du README suffisent à reproduire les points de contrôle A0 à B3.

PARTIE C Aller plus loin

BONUS OPTIONNEL · NON NÉCESSAIRE POUR TERMINER LE TP

LISEZ CECI AVANT DE COMMENCER

Cette partie est **entièrement facultative**. Les parties A et B permettent d'obtenir 20/20 sans elle, et aucune de ses pistes ne remplace une étape manquante des parties obligatoires. Ne l'abordez qu'une fois le point de contrôle B4 franchi.

C1 Refaire l'extraction avec Spoon et comparer

Refaites A2 et A3 (ou le graphe d'appel) avec Spoon 11.5.0, sur le même modèle de faits, puis comparez les résultats aux vôtres. Expliquez **au moins trois** différences de comptage et leur origine (par exemple : `CtField` et les constantes d'énumération, les appels implicites à `super()`, le mode *noClasspath*). L'annexe JDT/Spoon donne les chiffres mesurés.

C2 Visualiser

Exportez le graphe au format DOT ou GraphML et produisez une image lisible (par exemple avec Graphviz), éventuellement filtrée (un paquetage, les appelants d'une méthode). Une interface graphique est aussi possible ; elle ne doit être qu'une seconde façon d'afficher les résultats du même modèle.

C3 Affiner ou élargir l'analyse

Au choix, en justifiant l'intérêt et les limites : ajouter aux appels via une interface ou une classe abstraite les redéfinitions connues du projet (idée de la CHA, CM 02) sous forme d'une option ; ou analyser un projet libre de taille moyenne et commenter ce que les métriques et le graphe en révèlent.

5. Livrables et modalités de remise

Vous rendez **une seule archive par binôme**, nommée `NOM1_NOM2_HAI913I_TP1.zip` (en monôme : `NOM_Prenom_HAI913I_TP1.zip`), contenant les quatre livrables ci-dessous. Un livrable manquant, illisible ou inaccessible affecte directement les points correspondants du barème.

5.1. A — Compte-rendu (PDF)

Un **rapport d'ingénierie**, pas une documentation de code : il explique et justifie ce que vous avez fait. Ce n'est ni un listing des sources, ni de la Javadoc engendrée.

Page de titre : Université de Montpellier — Faculté des Sciences ; HAI913I — Évolution et restructuration des logiciels ; TP 1 — Analyse statique ; enseignant responsable : Bachar Rima ; année universitaire 2026–2027 ; date ; puis, **pour chaque membre** : prénom, nom, numéro étudiant et parcours (GL ou ICO).

Contenu attendu, précédé d'une table des matières :

1. **Objectifs et périmètre** : ce que fait votre outil, ce qu'il ne fait pas, les constructions Java prises en charge.
2. **Conception générale** : l'architecture, du chemin du projet à la sortie ; les choix de représentation du modèle de faits.
3. **Réponses étape par étape** (A0 à B4) : pour chacune, votre interprétation, la stratégie retenue, l'implémentation (extraits courts et ciblés), le résultat obtenu, la façon dont vous l'avez validé, les difficultés.
4. **Architecture et diagrammes** : uniquement ceux qui éclairent réellement votre solution (paquetages, classes du modèle, séquence d'une analyse…).
5. **Validation** : comparaison avec les points de contrôle, écarts et explications ; comportement sur les échantillons de robustesse.
6. **Difficultés et parties incomplètes**, et comment vous avez essayé de les adresser.
7. **Utilisation des outils d'intelligence artificielle** (§ 3.1) : si vous en avez utilisé ; lesquels (outil, modèle) ; pour quelles tâches ; des exemples représentatifs de l'aide obtenue ; comment vous l'avez vérifiée ; en quoi les décisions restent les vôtres. *Si vous n'en avez pas utilisé, écrivez-le explicitement.* Ne joignez pas d'historique complet de conversation.
8. **Regard critique** : limites de l'analyse statique menée (appels virtuels, réflexion, bibliothèques), pertinence des métriques, ce que vous amélioreriez.
9. **Conclusion**.
10. **Références** : notes de bas de page pour un renvoi isolé, bibliographie pour les sources substantielles (documentation, articles, code repris), dans un style de citation constant.

5.2. B — Code source

Le projet complet, qui se construit et s'exécute à partir des seules sources. **Sans** artefacts engendrés : pas de `target/`, `bin/` ni `out/`, pas de caches d'IDE ni de dépendances, pas de JAR, pas de fichiers temporaires ni de métadonnées système (`.DS_Store`, `__MACOSX`).

L'usage de **Git** est encouragé (un `.gitignore` adapté, des *commits* qui ont un sens, ni binaires ni identifiants). Si l'archive dépasse la limite affichée sur Moodle fournissez un lien vers un **dépôt Git** (GitHub, GitLab ou équivalent stable). **Si vous utilisez un dépôt, son URL figure dans le compte-rendu** et, de préférence, dans le README et `liens.txt`. Évitez les partages de fichiers génériques ou temporaires ; en cas d'hébergement externe, l'équipe pédagogique doit y avoir accès sans démarche et le lien doit rester valide pendant toute la correction.

5.3. C — README (PDF)

Rendu **en PDF** pour faire partie du dossier évalué (un `README.md` peut rester dans le dépôt). Un lecteur qui ne connaît pas votre travail doit pouvoir **reconstruire, exécuter et vérifier** l'outil à partir de ce seul document ; « ouvrir dans Eclipse et cliquer sur Run » ne suffit pas.

RUBRIQUE	CONTENU
Projet	modules et structure, architecture, entrée attendue, sorties produites, dépendances, approche JDT (et Spoon le cas échéant), limites connues
Environnement	version exacte du JDK, de Maven, de JDT Core (et de Spoon), système d'exploitation, **IDE et sa version, ou éditeur + outils en ligne de commande**
Construction	commandes exactes, depuis une copie fraîche
Exécution	commandes, arguments, exemples complets
Validation	projet d'entrée utilisé, sortie attendue ou points de contrôle
Dépannage	erreurs fréquentes et leur remède

5.4. D — Vidéo de démonstration

Une démonstration de l'**outil finalisé, en cours d'exécution** — pas de diaporama. Elle montre, dans l'ordre qui vous convient :

- la construction du projet depuis un état propre ;
- l'exécution sur le projet de validation : structure, appels et receveurs, métriques, seuil X et « 10 % », graphe d'appel ;
- l'usage de la ligne de commande (ou interface graphique) et au moins un cas d'erreur ;
- la correspondance entre ce qui est montré et les étapes du sujet.

Commentaire oral facultatif : des sous-titres ou du texte à l'écran conviennent. La vidéo n'a pas besoin d'être montée avec soin. Elle est incluse dans l'archive si sa taille le permet, placée dans le dépôt, ou déposée sur une plateforme stable (YouTube ou équivalent) ; son **URL figure alors dans le compte-rendu et dans le README ou `liens.txt`**. Vous êtes responsables des droits d'accès.

DÉPÔT ET DATE LIMITE

Structure attendue de l'archive (adaptez-la si votre architecture le justifie) :

```
NOM1_NOM2_HAI913I_TP1.zip          (monome : NOM_Prenom_HAI913I_TP1.zip)
|-- Compte-Rendu.pdf
|-- README.pdf
|-- source/                         pom.xml, src/ ... (sans target/, bin/, caches)
|-- video/
|   `-- demonstration.mp4           (ou liens.txt si la video est externe)
`-- liens.txt                       depot Git, video externe
```

DATE LIMITE — Consulter le dépôt Moodle « **TP1 — Analyse statique** ». Toute modification du calendrier sera répercutée sur ce dépôt.

Il existe **un seul espace de dépôt** pour tous les groupes de HAI913I. Un seul dépôt par binôme, effectué par l'un des deux membres ; les deux noms figurent dans le nom de l'archive et sur la page de titre.

6. Barème

Le barème porte sur les **parties A et B** et totalise **20 points**. Chaque critère renvoie aux étapes du sujet. Les correcteurs exécutent aussi votre outil sur un **second petit projet de même nature** que le projet de validation, pour vérifier que les résultats ne dépendent pas de ce dernier.

CRITÈRE	ÉTAPES	CE QUI EST REGARDÉ	PTS
A. CŒUR DE L'ANALYSE STATIQUE — 8 points			
Environnement, analyseur et parcours de l'AST	A0 A1	configuration JDT, visiteurs, rôle de `visit`	1
Extraction de la structure : classes, attributs, méthodes, héritage	A2	types comptés, visibilités, chaîne des superclasses	2
Extraction des appels et type statique du receveur	A3	*bindings* résolus, receveurs, non résolus signalés	2
Modèle de faits robuste, plusieurs unités de compilation	B1	indépendance vis-à-vis de JDT, résolution entre fichiers, déterminisme	1
Fonctionnement correct sur le projet de validation	A2 A3 B1	points de contrôle, second projet de même nature	2
B. MÉTRIQUES ET GRAPHE D'APPEL — 5 points			
Métriques de base	B2	questions 1 à 7 et 13, conventions respectées	2
Classements, seuil et cas limites	B2	questions 8 à 12 : arrondi, valeurs nulles, X	0,5
Construction du graphe d'appel selon la sémantique énoncée	B3	nœuds, arcs, poids, surcharges, externes et non résolus	2
Sortie exploitable et validation du graphe	B3	format lisible et justifié, appelants et appelées	0,5
C. QUALITÉ D'INGÉNIERIE ET REPRODUCTIBILITÉ — 2 points			
Architecture et séparation des responsabilités	B1 – B4	entrée, analyse, faits, métriques et graphe, sortie : séparés et testables	1
Construction et exécution reproductibles, gestion des erreurs	B4	copie fraîche, chemins invalides, projet vide, X mal formé	1
D. COMMUNICATION — 5 points			
Compte-rendu	—	structure, justification, validation, regard critique	2
Vidéo de démonstration	—	outil finalisé en exécution, lien avec les étapes	2
README	—	un tiers reconstruit, exécute et vérifie l'outil	0,5
Déclaration d'usage de l'IA, références et traçabilité	—	présente, honnête, cohérente ; sources citées	0,5
Total			20

PARTIE C — TRAITEMENT DES BONUS

Les bonus viennent **compenser** des points perdus sur les parties A et B. **Leur total est plafonné à +2 points, et la note finale à 20/20.**

BONUS	PTS
C1 — extraction avec Spoon fonctionnelle sur le projet de validation	**+0,5**
C1 — comparaison argumentée avec JDT (au moins trois différences expliquées)	**+0,5**
C2 — visualisation lisible du graphe, ou interface graphique branchée sur le même modèle	**+0,5**
C3 — affinement ou analyse d'un autre projet, justifiés et discutés	**+0,5**
Total pris en compte	**+2 max.**

Les bonus ne sont attribués que si l'outil fonctionne sur le projet de validation (points de contrôle A2, A3 et B2 franchis pour l'essentiel).

CE QUI N'EST PAS ÉVALUÉ

La mise en forme décorative du compte-rendu, le nombre de diagrammes, la longueur du rendu, le nombre de *commits*, le temps d'exécution, et le fait d'avoir ou non utilisé un assistant d'IA. À l'inverse, une incohérence entre le compte-rendu, le code et la vidéo affecte les critères correspondants.

7. Récapitulatif

À VÉRIFIER AVANT DE DÉPOSER

- ☐ Les points de contrôle A0 à B4 sont franchis, ou les écarts expliqués dans le compte-rendu.
- ☐ Les conventions de § 4 sont appliquées ; toute variante supplémentaire est nommée et définie.
- ☐ L'outil se construit depuis une copie fraîche, sans IDE, avec les commandes du README.
- ☐ La ligne de commande accepte le chemin du projet, l'action et le seuil X, et refuse proprement les entrées invalides.
- ☐ `Compte-Rendu.pdf` — page de titre des deux membres, table des matières, étapes, validation, section IA, regard critique, références.
- ☐ `README.pdf` — projet, environnement (JDK, Maven, JDT, IDE ou éditeur), construction, exécution, validation, dépannage.
- ☐ Vidéo montrant l'outil en exécution, accessible ; URL dans le compte-rendu si externe.
- ☐ URL du dépôt Git dans le compte-rendu, le cas échéant.
- ☐ Archive `NOM1_NOM2_HAI913I_TP1.zip`, sans `target/`, `bin/`, caches ni JAR.
- ☐ Dépôt dans l'espace Moodle « **TP1 — Analyse statique** », avant l'échéance qui y est affichée.